

■ Time & Space Complexity in Java

Complete Interview Reference Guide

Big-O Notation • Java Code Examples • DS Complexity Table • Tips

$O(1)$

$O(\log n)$

$O(n)$

$O(n \log n)$

$O(n^2)$

$O(2^n)$

Yuktha | Java Full Stack Development

1. What is Complexity Analysis?

Complexity analysis measures how an algorithm's **resource usage** grows as the input size (**n**) increases. It helps us compare algorithms independently of hardware or language.

Dimension	Measures	Java Context
Time Complexity	Execution steps / operations	Loop iterations, recursive calls
Space Complexity	Memory consumed	Stack frames, heap allocations

Why It Matters in Interviews

- Brute-force $O(n^2)$ vs optimised $O(n \log n)$ can be the difference between pass/fail.
- Interviewers expect you to state complexity before/after optimisation.
- Java collections have well-known complexities — know them cold.

■ *Always analyse both **Time** and **Space**. Don't forget recursive call stack space!*

2. Big-O Notation Reference

Big-O describes the **upper bound** (worst-case) growth rate. Other notations: Ω (Omega) = best case, Θ (Theta) = average case.

Notation	Name	Example Operation	Rating
$O(1)$	Constant	Array index access, HashMap get	■ Best
$O(\log n)$	Logarithmic	Binary search, BST lookup	■ Excellent
$O(n)$	Linear	Single loop, linear search	■ Good
$O(n \log n)$	Linearithmic	Merge sort, heap sort	■ Fair
$O(n^2)$	Quadratic	Nested loops, bubble sort	■ Poor
$O(n^3)$	Cubic	Triple nested loops, Floyd-Warshall	■ Bad
$O(2^n)$	Exponential	Recursive Fibonacci, subsets	■ Terrible
$O(n!)$	Factorial	Permutations, TSP brute force	■ Avoid

Simplification Rules

- **Drop constants:** $O(3n) \rightarrow O(n)$
- **Drop lower terms:** $O(n^2 + n) \rightarrow O(n^2)$
- **Different inputs = different variables:** $O(a + b)$, not $O(n)$
- **Multiplication rule:** nested loops multiply $\rightarrow O(n \times m)$

■ In Java, always count the **dominant term** and ignore coefficients.

3. Java Code Examples

$O(1)$ — Constant: Array Access

Time: $O(1)$ | Space: $O(1)$

```
int[] arr = {10, 20, 30};
int val = arr[1]; // Always 1 step
```

Direct index access. No loop, no recursion. Size of array doesn't matter.

$O(\log n)$ — Binary Search

Time: $O(\log n)$ | Space: $O(1)$ iterative / $O(\log n)$ recursive

```
int lo=0, hi=arr.length-1;
while (lo <= hi) {
    int mid = lo + (hi - lo) / 2;
    if (arr[mid] == target) return mid;
    else if (arr[mid] < target) lo = mid + 1;
    else hi = mid - 1;
}
return -1;
```

Halves search space each iteration. Works only on sorted arrays. $n=1M \rightarrow \sim 20$ steps.

$O(n)$ — Linear Search / Single Loop

Time: $O(n)$ | Space: $O(1)$

```
for (int i = 0; i < n; i++) {
    if (arr[i] == target) return i;
}
return -1;
```

One pass through the array. Steps grow linearly with input size.

$O(n \log n)$ — Merge Sort

Time: $O(n \log n)$ | Space: $O(n)$

```
void mergeSort(int[] arr, int l, int r) {
    if (l < r) {
        int m = (l + r) / 2;
        mergeSort(arr, l, m);
        mergeSort(arr, m+1, r);
        merge(arr, l, m, r); //  $O(n)$ 
    }
}
```

Divides array $\log n$ times, each level merges in $O(n)$. Best comparison-based sort.

$O(n^2)$ — Bubble Sort / Nested Loops

Time: $O(n^2)$ | Space: $O(1)$

```
for (int i = 0; i < n; i++)
    for (int j = 0; j < n-i-1; j++)
        if (arr[j] > arr[j+1])
            swap(arr, j, j+1);
```

Two nested loops each running n times $\rightarrow n \times n$ operations. Avoid for large n .

$O(2^n)$ — Recursive Fibonacci

Time: $O(2^n)$ | Space: $O(n)$ stack depth

```
int fib(int n) {  
    if (n <= 1) return n;  
    return fib(n-1) + fib(n-2);  
}
```

Each call branches into 2 → exponential. Use memoization to reduce to $O(n)$.

4. Java Collections — Complexity Cheat Sheet

Data Structure	Access	Search	Insert	Delete	Space
Array / int[]	$O(1)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$
ArrayList	$O(1)$	$O(n)$	$O(1)^*$	$O(n)$	$O(n)$
LinkedList	$O(n)$	$O(n)$	$O(1)^{**}$	$O(1)^{**}$	$O(n)$
Stack (ArrayDeque)	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$
Queue (ArrayDeque)	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$
HashMap	$O(1)^*$	$O(1)^*$	$O(1)^*$	$O(1)^*$	$O(n)$
TreeMap	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n)$
HashSet	—	$O(1)^*$	$O(1)^*$	$O(1)^*$	$O(n)$
TreeSet	—	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n)$
PriorityQueue (min)	$O(1)$ peek	$O(n)$	$O(\log n)$	$O(\log n)$	$O(n)$
Binary Search Tree	$O(\log n)^*$	$O(\log n)^*$	$O(\log n)^*$	$O(\log n)^*$	$O(n)$

* Amortized average case | ** With known node reference

Quick Rules for Interviews

- Use **HashMap/HashSet** when you need $O(1)$ lookup — most frequent pattern.
- Use **TreeMap/TreeSet** when sorted order matters → $O(\log n)$.
- Prefer **ArrayDeque** over Stack class (legacy) for stack/queue.
- **ArrayList** add-to-end is $O(1)$ amortized; add-in-middle is $O(n)$.
- **PriorityQueue** peek is $O(1)$, poll/add are $O(\log n)$.

5. Sorting Algorithms Complexity

Algorithm	Best	Average	Worst	Space	Stable?
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No
Counting Sort	$O(n+k)$	$O(n+k)$	$O(n+k)$	$O(k)$	Yes
Radix Sort	$O(nk)$	$O(nk)$	$O(nk)$	$O(n+k)$	Yes
Arrays.sort()	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(\log n)$	Yes*

* Arrays.sort() uses TimSort for objects (stable), dual-pivot quicksort for primitives.

Java Interview Tip

- Arrays.sort(arr) → primitives: $O(n \log n)$, not stable.
- Arrays.sort(arr, comparator) → objects: TimSort, stable.
- Collections.sort(list) → TimSort, $O(n \log n)$, stable.
- For nearly-sorted data → Insertion Sort wins. For large random data → Merge/Quick.

6. Space Complexity Deep Dive

Space complexity = **Auxiliary Space** (extra space used by algorithm) + Input Space. In interviews, we usually measure **auxiliary space** only.

Recursion Stack Space

```
// O(n) space - n frames on call stack
int factorial(int n) {
    if (n == 0) return 1;
    return n * factorial(n - 1);
}
```

- Each recursive call adds a **stack frame** → $O(\text{depth})$ space.
- Binary recursion (like Fibonacci) → $O(n)$ stack depth, not $O(2^n)$ space.

In-place vs Extra Space

Pattern	Space	Java Example
Two pointers on array	$O(1)$	Reverse, palindrome check
Sliding window	$O(1)$	Max subarray, substring
HashMap for frequency	$O(n)$	Two Sum, anagram check
Recursive DFS	$O(h)$	h = tree height
BFS with queue	$O(w)$	w = max queue width
DP table (2D)	$O(n \times m)$	LCS, Edit Distance
DP optimised (1D)	$O(n)$	0/1 Knapsack space opt.

Common Space Patterns

```
// O(1) Space - Two Pointer
int l=0, r=arr.length-1;
while(l < r) { swap(arr,l++,r--); }

// O(n) Space - Memoization
Map memo = new HashMap<>();
int fib(int n) {
    if (memo.containsKey(n)) return memo.get(n);
    if (n<=1) return n;
    memo.put(n, fib(n-1)+fib(n-2));
    return memo.get(n);
}
```

■ *Trading time for space (memoization): Fibonacci goes from $O(2^n)$ time to $O(n)$ time, $O(n)$ space.*

7. Interview Tips & Patterns

1. State complexity upfront

After understanding the problem, say: 'My brute force is $O(n^2)$ time, $O(1)$ space. I can optimise to $O(n)$ using a HashMap.'

2. Identify the bottleneck

If you have a sort + loop: $O(n \log n) + O(n) = O(n \log n)$. Dominant term wins.

3. Loops × Loops = Multiply

Nested loop over same array = $O(n^2)$. Inner loop from i to n : still $O(n^2)$ amortized.

4. Recursion depth = space

DFS on tree: $O(h)$ space. Balanced BST $h = \log n \rightarrow O(\log n)$. Skewed $h = n \rightarrow O(n)$.

5. Hash table tradeoff

Switching from array search $O(n)$ to HashMap $O(1)$ is the most common optimisation pattern.

6. String operations in Java

String concatenation in loop: $O(n^2)$ — use `StringBuilder`. `charAt()` = $O(1)$. `substring()` = $O(n)$.

7. Collections.sort vs custom

Always mention Java's built-in sort is $O(n \log n)$ and whether you'd use it or write custom.

8. Amortized analysis

`ArrayList add()` is $O(1)$ amortized because resizing is rare. Mention 'amortized' to impress.

8. Quick Reference Card

Complexity Hierarchy (Best → Worst)

$O(1) < O(\log n) < O(\sqrt{n}) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$

One-Liner Rules

Single loop	$O(n)$
Nested loop (same input)	$O(n^2)$
Loop halving each iter	$O(\log n)$
Divide & conquer + merge	$O(n \log n)$
Recursion branching 2	$O(2^n)$ time, $O(n)$ space
HashMap get/put	$O(1)$ average
TreeMap get/put	$O(\log n)$
Arrays.sort()	$O(n \log n)$
String + in loop	$O(n^2)$ — use StringBuilder
StringBuilder append	$O(1)$ amortized

■ *In an interview: always mention best/average/worst case AND space complexity.*